

El carrito se hace global

Context y el primer hook propio. El refactor a arquitectura hexagonal. Y la tienda por fin sabe quién eres: login con un JWT real.

4 h 15 · L-M-V

Retomamos donde cortamos

STEER
RC
LA

SIN REPASOS LARGOS: RETOMAMOS Y SEGUIMOS

El plan de hoy

0. Reapertura

8'

el punto de corte + tareas destacadas

A. Remate de la clase 5

0-75'

solo lo pendiente · guion de la clase 5

B. Context + useCart

83'

el carrito global · la caída de las props

C. Arquitectura hexagonal

45'

refactor guiado: dominio · infraestructura · aplicación · ui

D. La sesión nace

58'

JWT · auth.ts · SesionContext · /login

F. Cierre + 5 tareas

20'

push demostrado, dudas y anticipo

**Receso de 15 min** alrededor de las 2:00 · verde = **el corazón del día** · se corta a las 3:50 donde toque

EL PROBLEMA CON NOMBRE

Prop drilling

App → **Layout** → Header

el Layout no usa el resumen: solo lo carga

App → **Home** → ProductCard

Home no agrega nada: transporta onAgregar

App → Detalle

otra copia de onAgregar

App → Checkout

carrito, total, onVaciar... y la cadena crece con cada página

Pasar una prop por componentes que no la usan, solo para que llegue al que sí la usa. Los rojos **solo la transportan**.

React tiene una pieza para datos que son de TODOS: **Context** — el de arriba guarda el dato y el de abajo lo lee directo.

TRES PIEZAS Y YA

Las piezas de Context

createContext

crea el **contexto**. Va fuera de todo componente: se crea UNA vez, a nivel de módulo.

<Provider value>

el que **entrega el valor**. Envuelve a los componentes (en main) y lo deja disponible: items, total, agregar...

useContext

el que **lee el valor**. Cuarto hook del curso: desde cualquier componente de adentro, a cualquier profundidad.

```
export function useCart() {  
  const contexto = useContext(CarritoContext);  
  if (!contexto) throw new Error('useCart solo funciona dentro de  
<CarritoProvider>');  
  return contexto; // tu primer hook propio: lee el contexto y comprueba que exista  
}
```

Regla de decisión: Context es para el estado de TODA la aplicación (carrito, sesión, tema). Lo local — el buscador, un panel abierto, los productos que App pide — se queda donde vive. **Comparte lo mínimo.**

PARA RECONOCERLOS SIN SUFRIRLOS — NINGUNO SE CODEA

Errores frecuentes de Context

1 • consumir fuera del Provider

recibes el valor inicial (null) y algo revienta lejos del culpable — el `if` de `useCart` lo convierte en un mensaje exacto

2 • olvidar envolver en `main`

el mismo síntoma que el anterior; el mismo `if` te avisa qué falta

3 • el contexto gigante

carrito + productos + filtros + sesión en UN contexto: cada tecla del buscador repinta a TODOS los consumidores → un contexto por asunto

4 • `createContext` dentro del componente

se crea un contexto nuevo en cada render y los componentes dejan de recibir el valor → siempre afuera, a nivel de módulo

¿DÓNDE VA CADA COSA? · TU PROYECTO YA VIVE ASÍ

Arquitectura hexagonal

INFRAESTRUCTURA

`api.ts · datos.ts`
`pedidos.ts · almacen.ts ·`
`auth.ts`

conecta con la red y el disco. Los nombres en inglés de la API solo existen aquí

DOMINIO

`tipos.ts · carrito.ts`

las reglas de la tienda: funciones que reciben datos y devuelven datos. Ni React, ni red, ni disco

UI

`components/ · pages/`
`layout/ · formato.ts`

lo que se ve. Consume la aplicación; nunca toca el disco directo

APLICACIÓN

`CarritoContext.tsx · useCart.ts · SesionContext.tsx · useSesion.ts`

los casos de uso, con React: conectan dominio e infraestructura con la UI

Hoy no cambiamos el código: le ponemos **nombre y carpeta** a lo que ya era cierto. La misma arquitectura que en el proyecto final vale **un punto extra**.

DESPUÉS DE LA MUDANZA · BUILD VERDE

El mapa de tu proyecto

```
src/  
  dominio/           tipos.ts · carrito.ts ← no importa de nadie  
  infraestructura/   api.ts · datos.ts · pedidos.ts · almacen.ts · auth.ts ← importa del dominio  
  aplicacion/        CarritoContext · useCart · SesionContext · useSesion ← importa dominio + infra  
  ui/                components/ · pages/ · layout/ · formato.ts ← importa de todos  
App.tsx · main.tsx · index.css
```

La regla de dependencia: las capas de afuera importan del dominio, nunca al revés. Si `carrito.ts` quiere importar `React` o `api.ts`, algo está en el lugar equivocado. Refactorizar = cambiar la forma sin cambiar el comportamiento: se parte de build verde y se llega a build verde.

LA TIENDA POR FIN SABE QUIÉN ERES

JWT: el token de sesión

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJlbWlseXMiLCJmaXJzdE5hbWUiOiJFbWlseSIsImV4cCI6MTc1NTY0ODAwMH0.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

header

qué algoritmo firmó el token

payload

tus datos impresos: id, nombre, **vencimiento**. Cualquiera puede leerlo: **no está cifrado**

firma

solo el servidor sabe hacerla. Cambia una letra y no cuadra: **nadie lo falsifica**

El servidor lo emite cuando demuestras quién eres (POST /auth/login). Desde entonces, cada petición privada lleva **el token en el header**. Y como toda sesión, **vence**.

PARA RECONOCERLOS SIN SUFRIRLOS — NINGUNO SE CODEA

Errores frecuentes de sesión

1 • guardar la contraseña

jamás, en ningún lado, ni en el estado más tiempo del necesario: se envía y se olvida. Lo que se guarda es el **token**

2 • creer que el token está cifrado

está FIRMADO: cualquiera lo lee (pégalo en jwt.io). Nada secreto adentro

3 • el token eterno

todo token vence. La próxima clase el nuestro programa su propio cierre

4 • el token en la URL

las URLs quedan en historiales y logs. Viaja en el header `Authorization: Bearer` — lo estrenamos con `/auth/me`

HASTA DONDE LLEGAMOS HOY

Lo de hoy

prop drilling → Context

createContext crea el contexto, el Provider entrega el valor, useContext lo lee — para el estado de todos

useCart

tu primer hook propio: lee el contexto y comprueba que el Provider exista

la caída de las props

el que necesita, consume; nadie transporta. El Layout quedó sin props

hexagonal

dominio puro al centro, adaptadores en los bordes; las flechas apuntan hacia adentro

JWT

un token firmado: se puede leer, no se puede modificar, y vence. Nunca guarda secretos

useNavigate

cuando navegar es consecuencia del código, no un clic del usuario

CRITERIO DE ENTREGA: NPM RUN BUILD SIN ERRORES + PUSH

5 tareas

FÁCIL

Replica la clase, hasta donde llegamos, con la grabación al lado. Es LA tarea.

INTERMEDIO

formato.ts, ¿presentación o dominio? El comentario argumentado arriba del archivo; si crees que es dominio, muévelo y defiéndelo.

INTERMEDIO

La página /pedidos con las capas: vive en ui/pages, lee con leerPedidos() de infraestructura, jamás toca localStorage directo.

DIFÍCIL

Favoritos globales: FavoritosContext + useFavoritos calcados del carrito; corazón en la tarjeta, contador en el header, persistencia en almacen.ts comprobando al leer.

DIFÍCIL

La pregunta: pega /checkout sin sesión. Entra cualquiera. ¿Dónde pondrías el control y qué harías con el que no entró? Tu plan, en un comentario.

Próxima clase — retomamos donde cortamos

¿Quién vigila `/checkout`?

```
// hoy: F5 y la sesión muere • /checkout recibe a cualquiera
GET /auth/me  Authorization: Bearer <token> ← el token, validado por el
servidor
<Route element={<RutaProtegida />}} ... </Route>
```

La sesión se vuelve seria: el token sobrevive al F5 (guardado y comprobado al leer), se valida contra el servidor, **expira solo**, y las rutas privadas ganan **RutaProtegida**. Después, la tienda aprende a no repintar de más: **memo**, **useMemo** y **lazy**.

Trae las tareas hechas y el build en cero. Nos vemos.